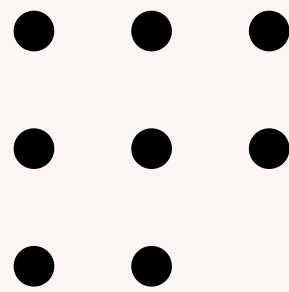
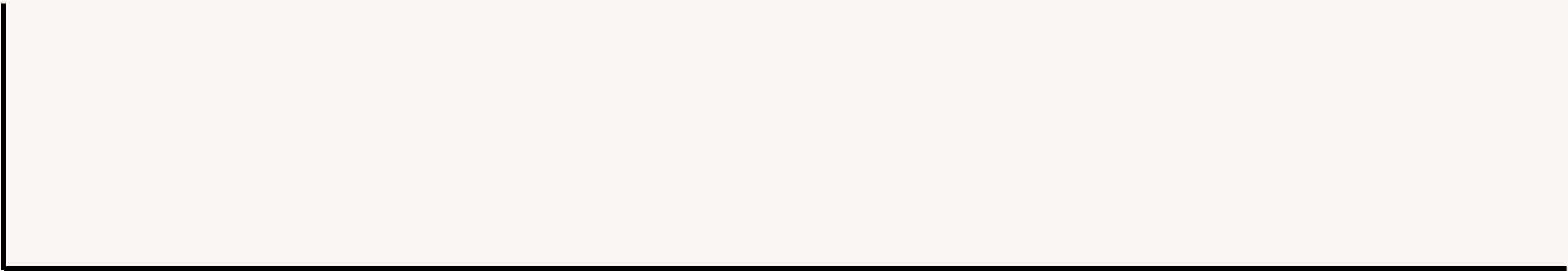


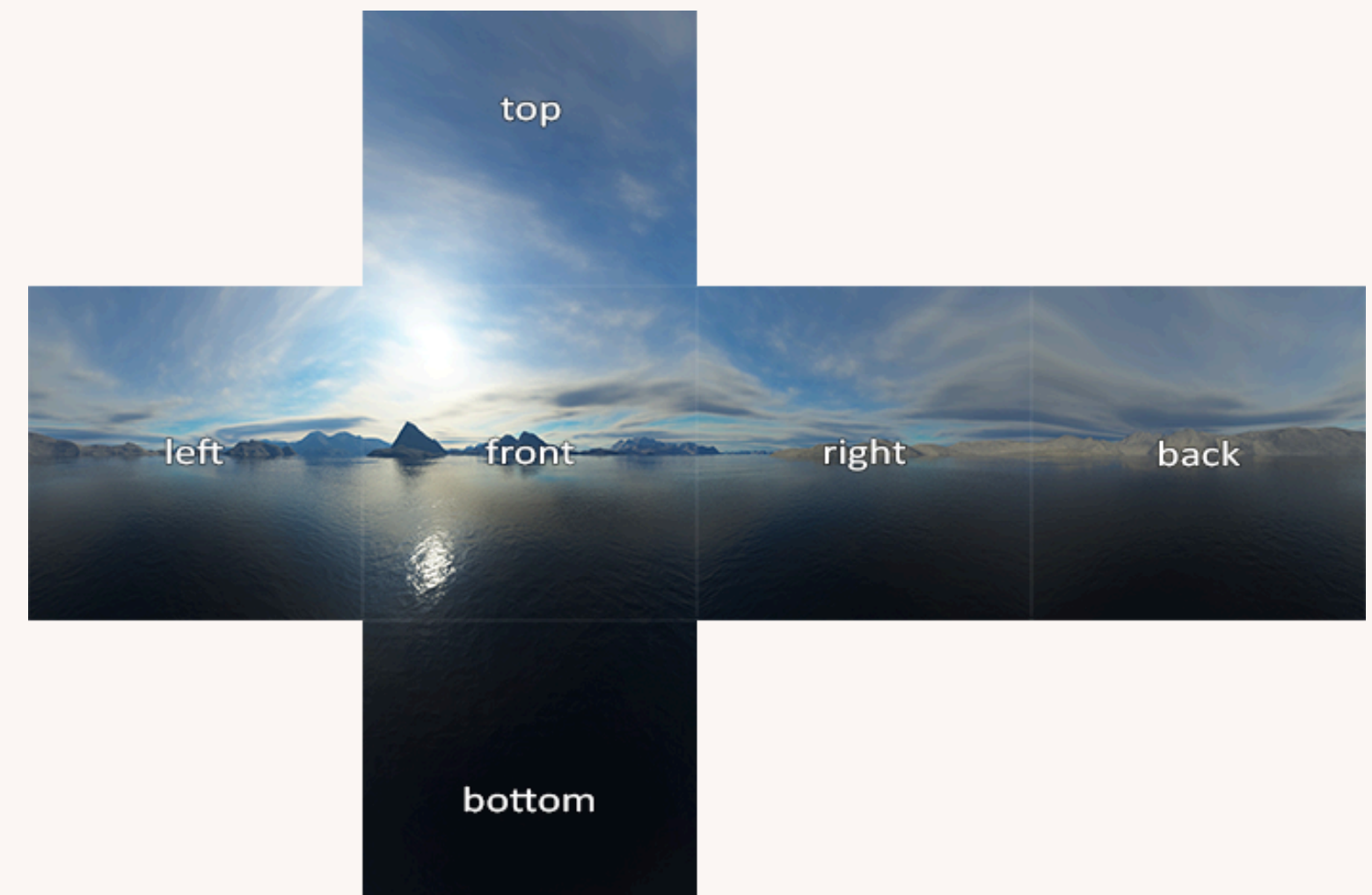
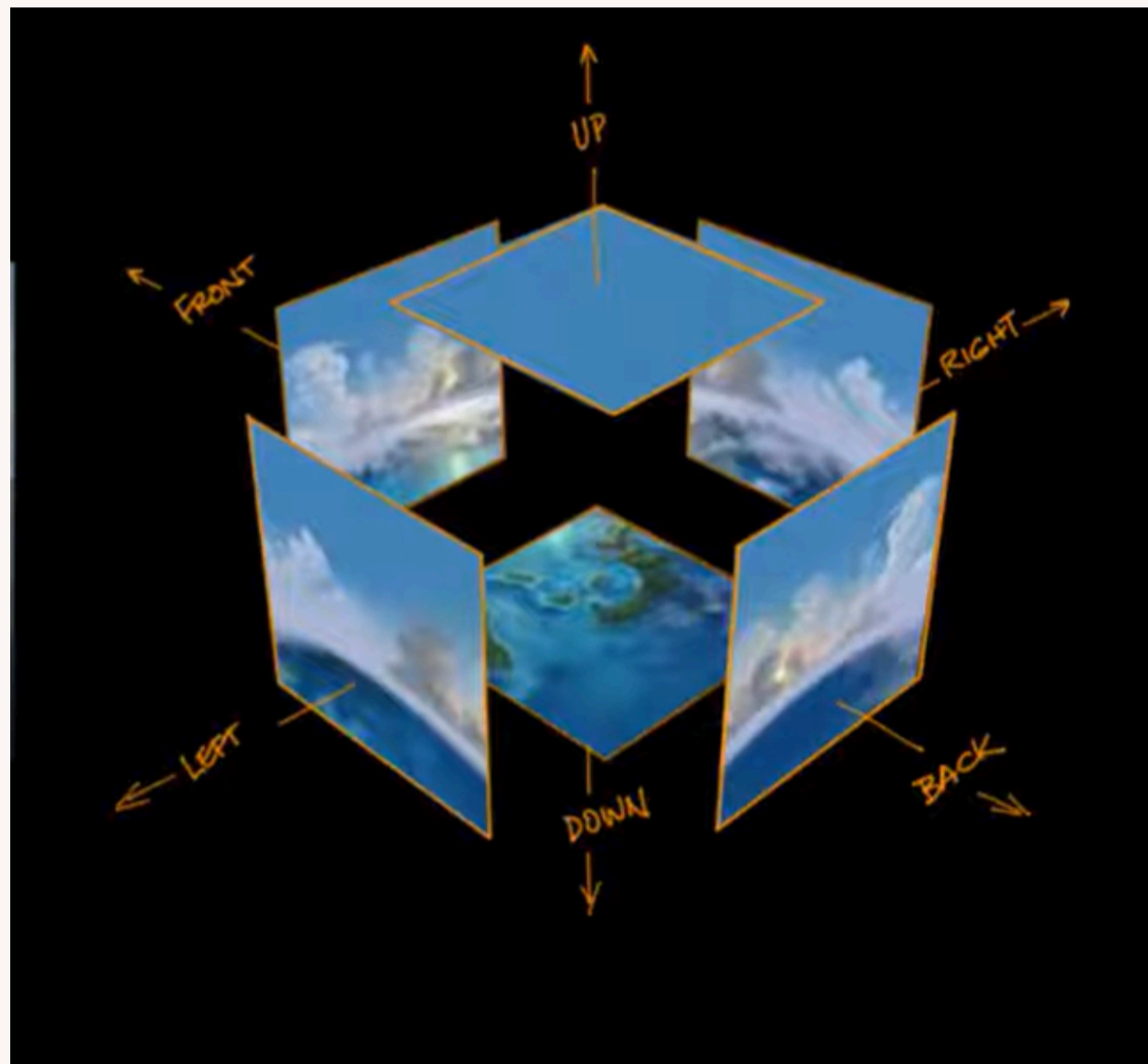
TEMA 1 : ENVIRONMENT MAPPING

Abrahão Francis e Gabriel Ponzoni



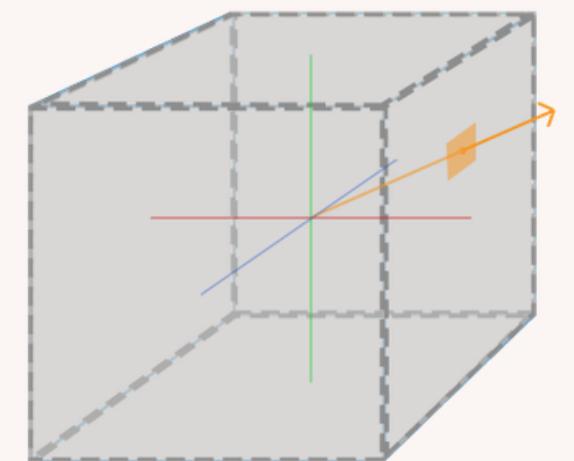
CUBEMAP

O cubemap é gerado através de seis imagens distintas, capturadas a partir de um mesmo ponto fixo em seis direções (frente, trás, esquerda, direita, cima e baixo), cada uma com um campo de visão de 90 graus.



Essas texturas são coladas uma nas outras a estrutura e ordem ocorre em normalmente em formato de cruz.

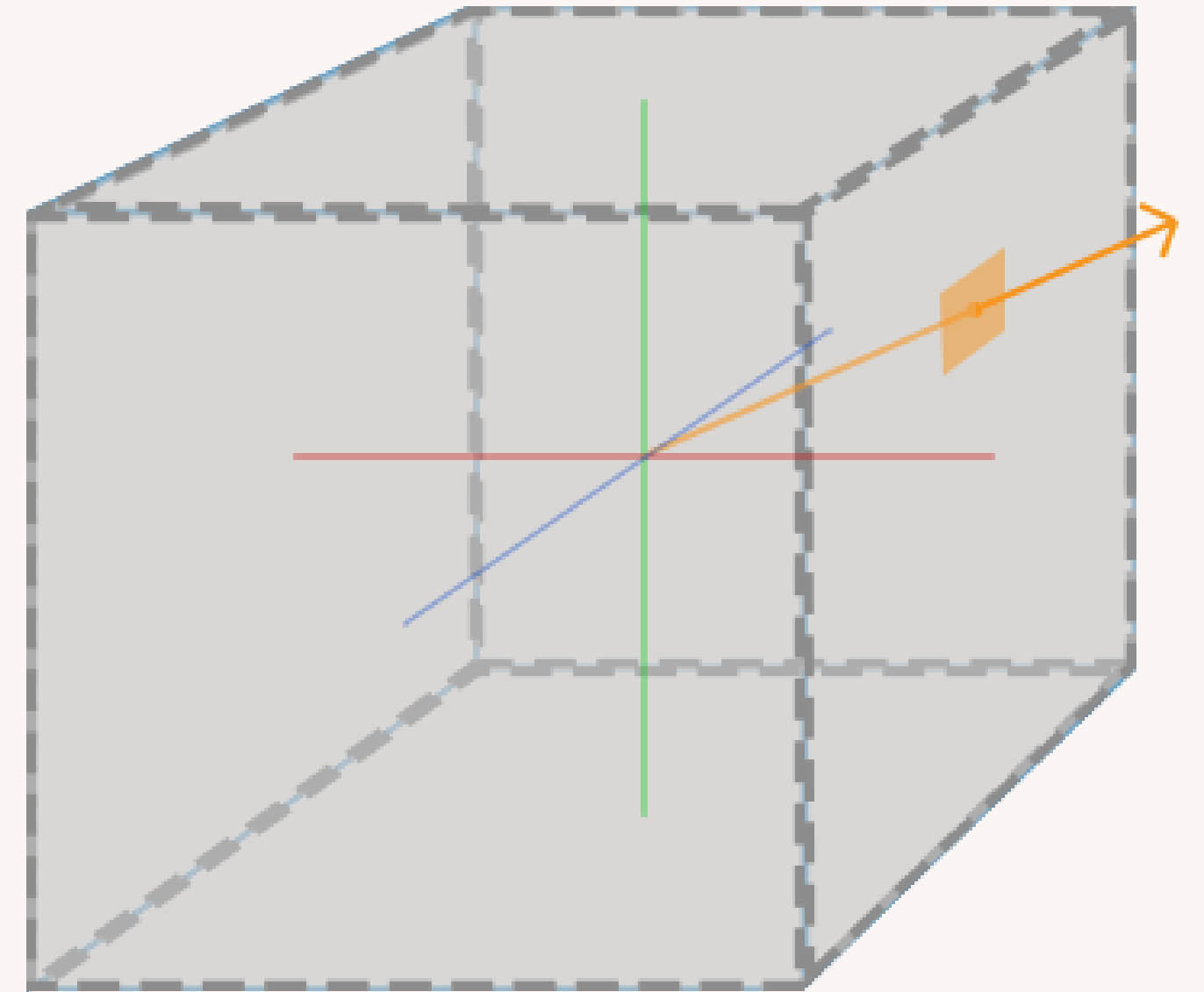
Diferente de uma textura 2D comum, que usa coordenadas U e V, o cubemap é amostrado utilizando um vetor 3D (X, Y, Z). O motor gráfico calcula qual das seis faces deve ser amostrada com base na direção desse vetor.



CUBEMAP

Passo 1 (Major Vector): Precisamos selecionar qual das 6 imagens mostrar

- Pra isso usamos a regra do Major Vector, olhando qual foi o maior valor X ou Y ou Z.
- Depois de ver qual o valor de maior valor verifica se o **sinal** é **positivo** ou **negativo** daquele número para escolher a parede. Por exemplo, se o nosso vetor for $(-0.8, 0.2, 0.1)$, o maior valor absoluto é o **0.8** no eixo X. Como o número original era negativo, a GPU sabe na mesma hora que o laser não foi para a direita, ele foi para a esquerda, atingindo a textura da face `NEGATIVE_X`.



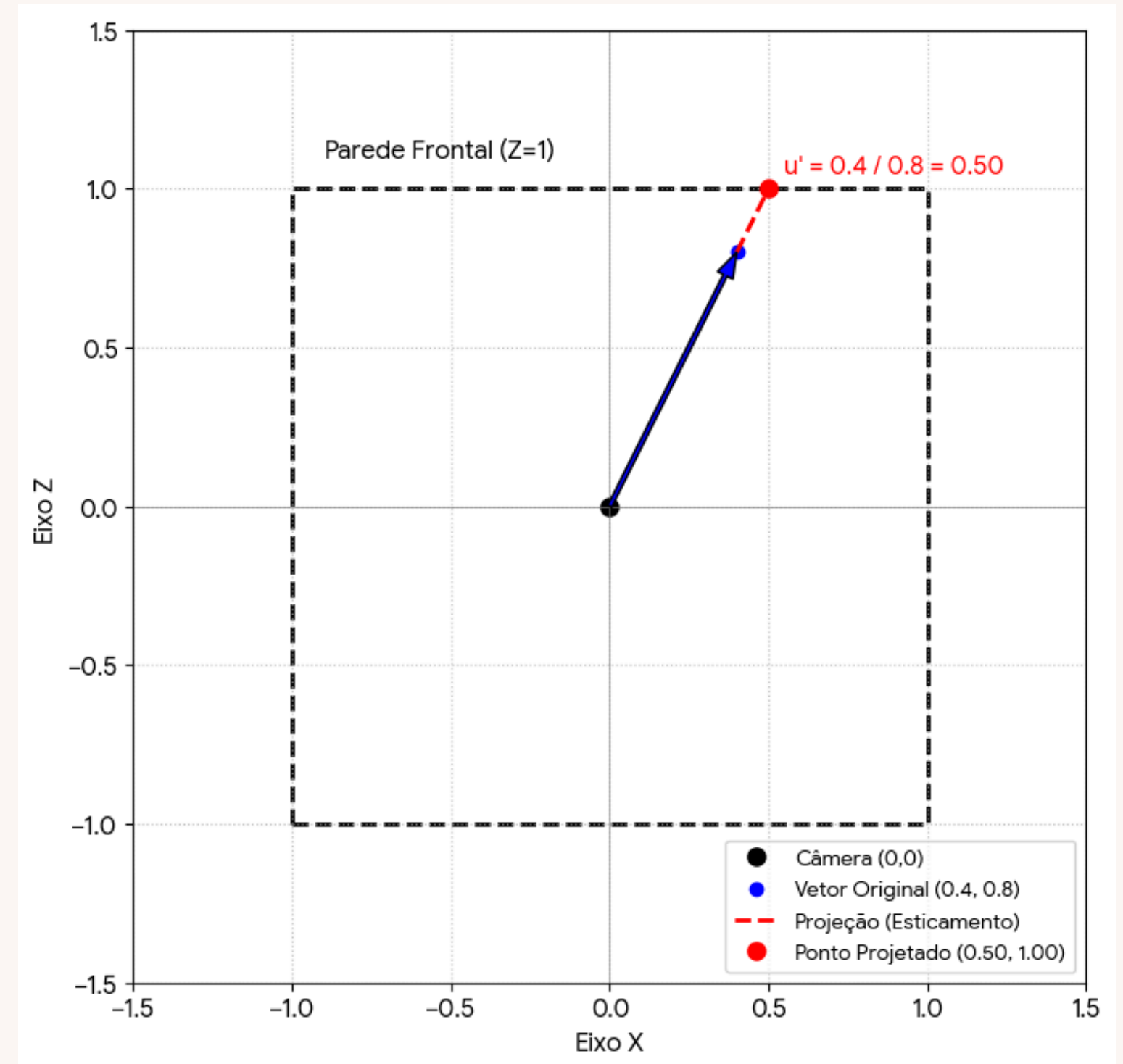
CUBEMAP

Passo 2 (Projeção): Precisamos descobrir a posição exata do impacto dentro da face do plano 2D. Ex:

$(-0.8, 0.2, 0.1)$

- Pegamos as outras duas coordenadas e dividimos pelo valor absoluto da maior delas, gerando u' e v' . No exemplo, dividimos o 0.2 e o 0.1 por 0.8 .
- Fazemos isso porque a coordenada maior dividida por ela mesma resulta em 1 (esticando o raio até encostar na parede). A mesma proporção de esticamento é aplicada aos outros valores.
- O resultado gera um plano cartesiano local que varia exatamente entre -1 e +1 dentro daquela face 2D.

CUBEMAP VISTO DE CIMA (EXCLUI EIXO Y)



CUBEMAP

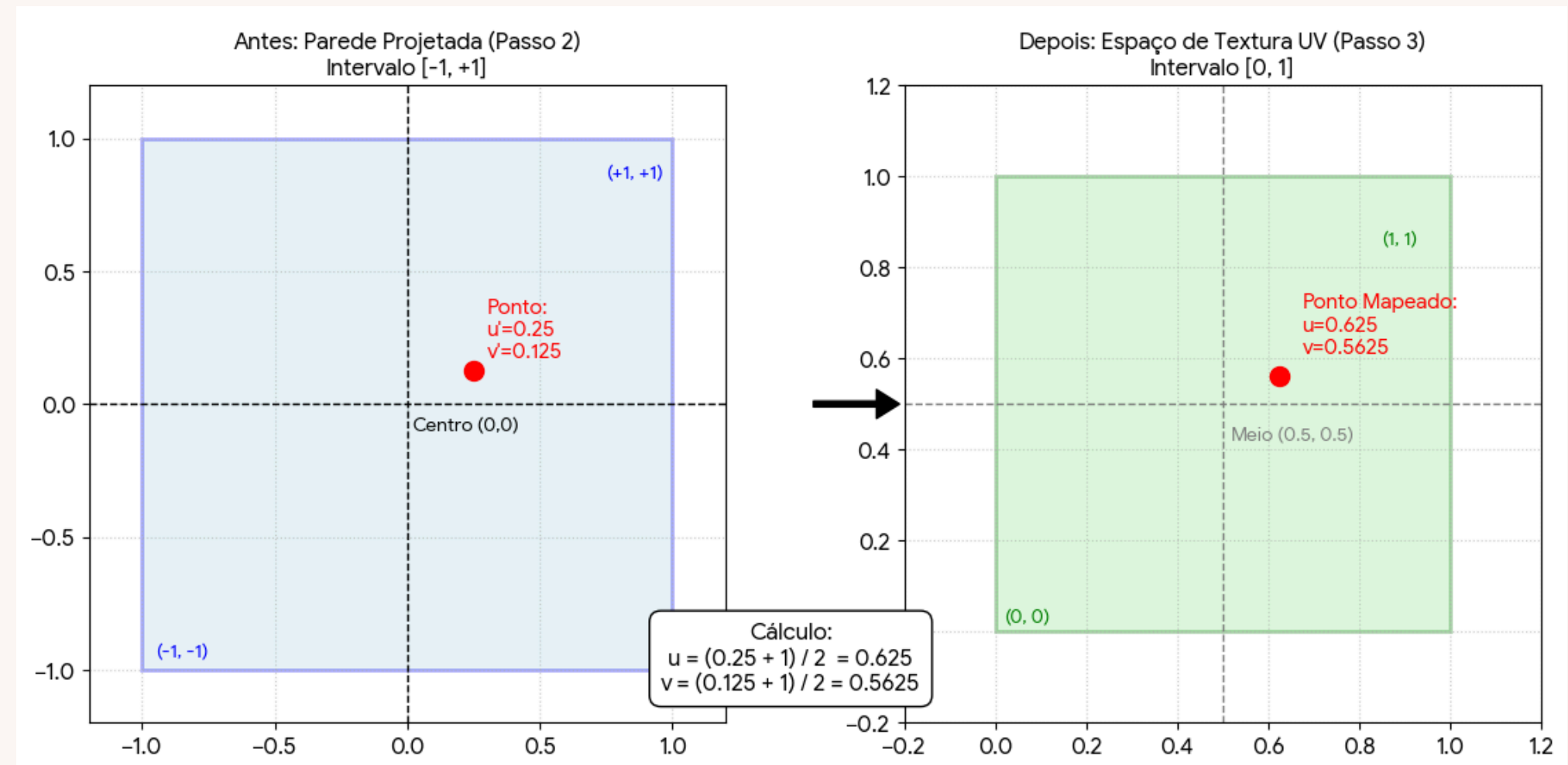
Passo 3 (Remapeamento): Remapeamento do espaço de Textura UV. Ex: $u' = 0.25$ e $v' = 0.125$

- Texturas não possuem coordenadas negativas, então puxamos tudo para cima com +1 e dividimos tudo por 2.

$$u = \frac{u' + 1}{2} \quad \text{e} \quad v = \frac{v' + 1}{2}$$

- Dividimos por 2 para comprimir tudo para dentro do intervalo de 0 a 1
- O resultado gera um plano cartesiano local que varia exatamente entre 0 e 1 dentro daquela face 2D.

CUBEMAP VISTO DE FRENTE PARA A FACE SELECIONADA

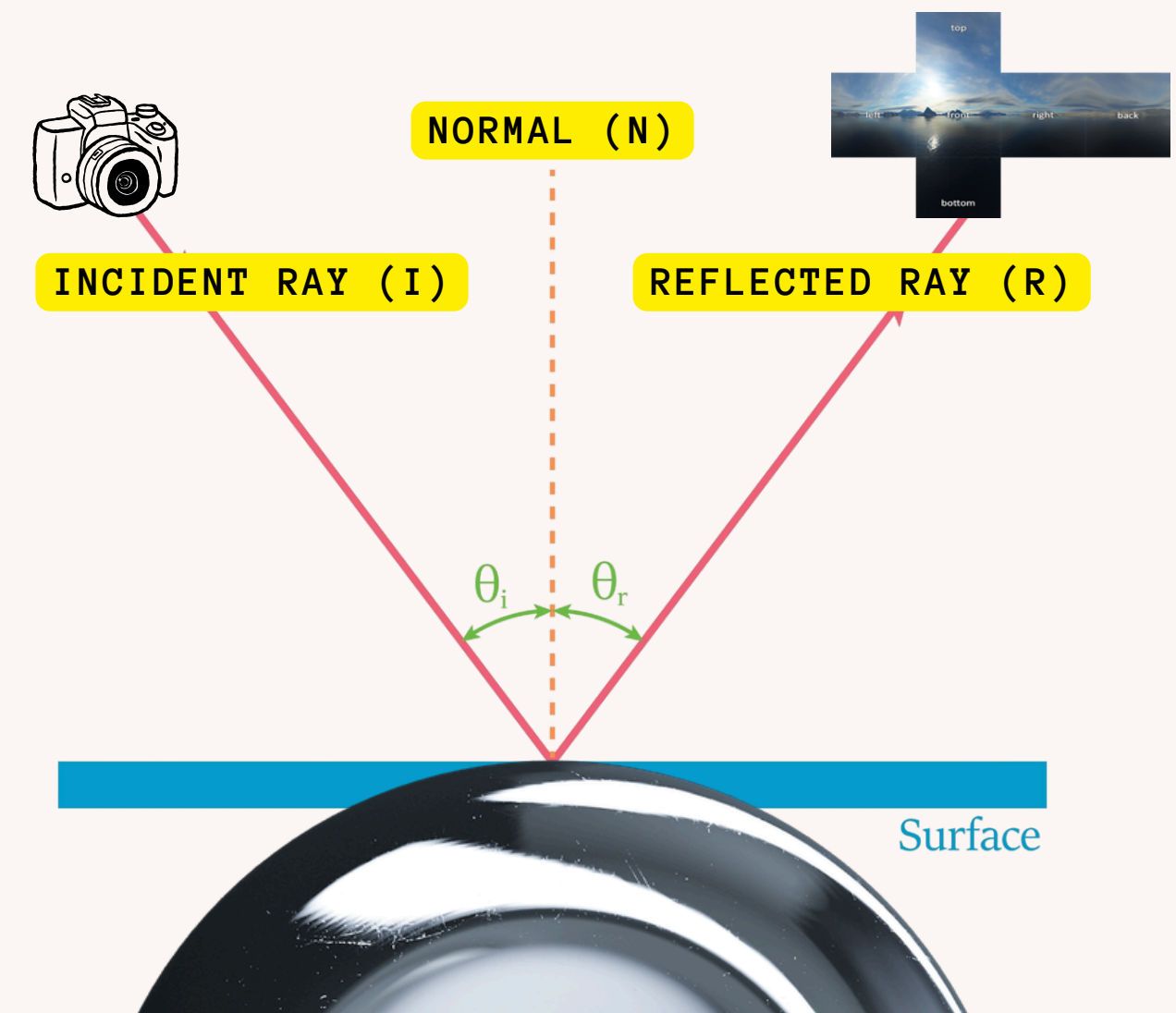


O OBJETO ESPECULAR

A reflexão é calculada a partir de um vetor que parte da câmera (Vetor I) e atinge a superfície do objeto, que possui uma inclinação (Normal N).

- Usando a função **reflect(I, N)** o resultado desses dois é o vetor reflexão (R)
- O vetor R nos dá a direção exata do céu que o observador está vendo refletir naquela face do objeto.
- Quando o vetor R bate na parede da Skybox, a placa de vídeo roda exatamente os mesmos 3 passos explicados anteriormente (Major Vector, Projeção e Remapeamento)

DIAGRAMA



DECISÕES

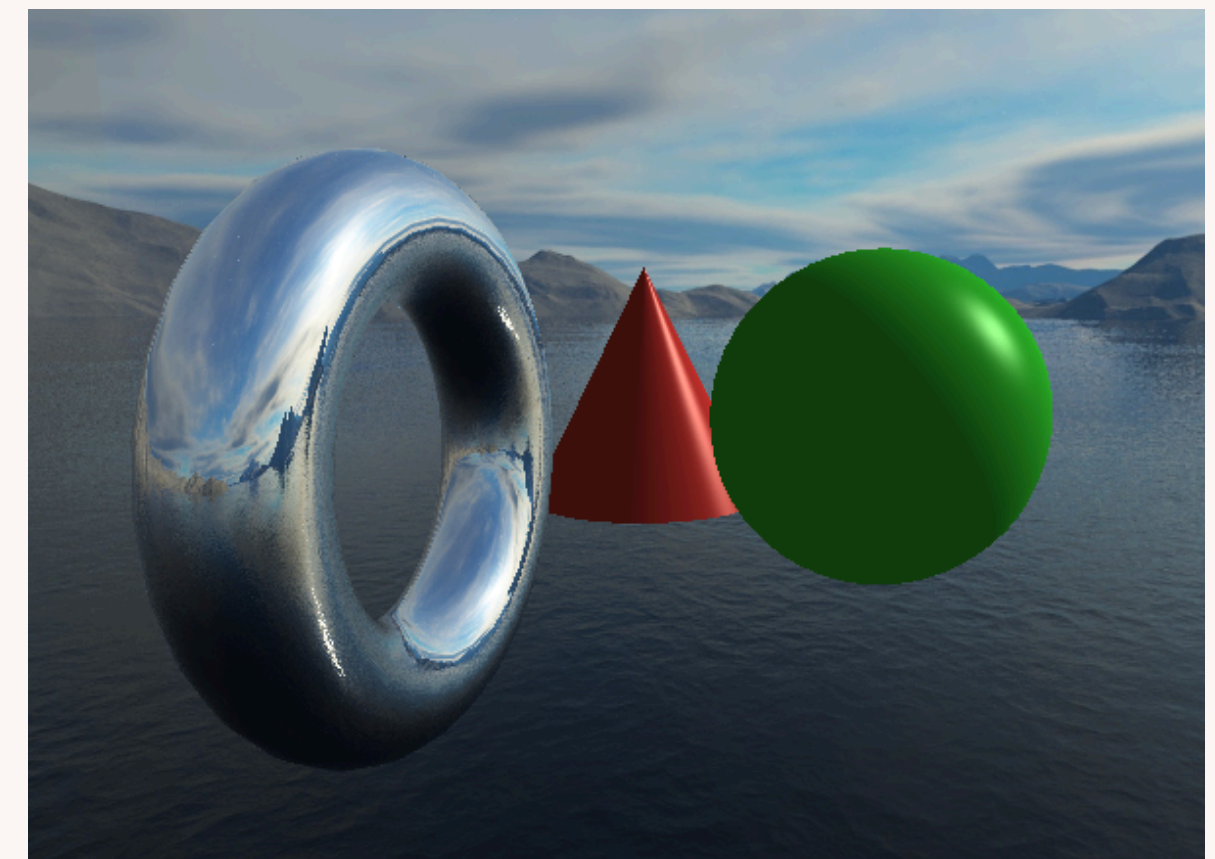
Escolhemos desenvolver o projeto utilizando:

- **Skybox Estático:** Mantivemos a implementação apresentada nos exemplos da Aula 9 sobre skybox. A escolha foi feita por ser uma solução mais simples de implementar, já que já conhecíamos seu funcionamento.
- **Modelagem Paramétrica:** Decidimos gerar o Torus, o Cone e a Esfera por meio de equações matemáticas diretamente no código. Isso facilitou o mapeamento das coordenadas dos vértices e das normais dos objetos.
- **Shaders:** A lógica utilizada foi a mesma dos exemplos vistos em aula, onde há um shader para o skybox e outro para os objetos. O fator reflexivo é definido por um pseudo enum chamado modo: se estiver configurado como fosco, o objeto fica totalmente fosco; se estiver configurado como reflexivo, o objeto fica totalmente reflexivo.

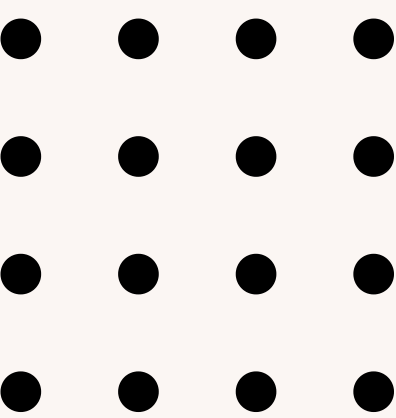
```
# --- Cone Fosco (Vermelho) ---
vao_cone, n_indices_cone = Objetos_cenario["cone"]
glBindVertexArray(vao_cone)
glUniform1i(glGetUniformLocation(Shader_objeto, "u_mod"), MODO_FOSCO)
glUniform3f(glGetUniformLocation(Shader_objeto, "objectColor"), 0.8, 0.2, 0.2)
transformacaoGenerica(Shader_objeto, -2.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0)
glDrawElements(GL_TRIANGLES, n_indices_cone, GL_UNSIGNED_INT, None)

# --- Esfera Fosca (Verde) ---
vao_esfera, n_indices_esfera = Objetos_cenario["esfera"]
glBindVertexArray(vao_esfera)
glUniform1i(glGetUniformLocation(Shader_objeto, "u_mod"), MODO_FOSCO)
glUniform3f(glGetUniformLocation(Shader_objeto, "objectColor"), 0.2, 0.8, 0.2)
transformacaoGenerica(Shader_objeto, 0.0, 0.0, -2.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0)
glDrawElements(GL_TRIANGLES, n_indices_esfera, GL_UNSIGNED_INT, None)

# --- Torus Reflexivo ---
vao_torus, n_indices_torus = Objetos_cenario["torus"]
glBindVertexArray(vao_torus)
glUniform1i(glGetUniformLocation(Shader_objeto, "u_mod"), MODO_REFLEXIVO)
# objectColor é ignorado no modo reflexivo, mas pode deixar qualquer valor
transformacaoGenerica(Shader_objeto, 2.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0)
glDrawElements(GL_TRIANGLES, n_indices_torus, GL_UNSIGNED_INT, None)
```



Comparação de objetos em cena



Obrigado!

